

# "brainimg: A Reproducible Systems Study of Generative-Recall Image Compression"

Pedro A. Gracia Fajardo

2026-06-19

## Abstract

|

## 1. Introduction

Every widely used image format answers a variant of the same question: how do we store these pixels efficiently? PNG stores exact colors and predicts the next pixel; JPEG/WebP discard transform coefficients the human visual system barely notices; AVIF generalizes this to AV1 intra-frames. All three paradigms share two properties: the file is bound to a native resolution, and "loss" manifests as visible artefacts—blockiness, ringing, colour bleeding.

Neuroscience offers a different model. The retina compresses roughly 130 million photoreceptor signals into about one million optic-nerve fibres (a  $\sim 130:1$  ratio) by transmitting edges and contrast rather than absolute brightness. Deeper in the visual cortex, representation becomes increasingly semantic—neurons respond to faces, objects, materials—until recall itself becomes a generative act: to imagine an apple is not to "open a file" but to drive top-down signals from concept areas back into the visual cortex and hallucinate the sensory experience back into existence. The brain stores meaning and the rules to rebuild, not a pixel grid.

This paper is a reproducible systems study of a compression paradigm that mimics that idea. We ask:

**Can a modern diffusion model serve as the decoder for a kilobyte-scale image format, conditioned on a text caption and a handful of low-resolution structural maps?**

We do **not** claim a new state of the art, nor that the result replaces JPEG. We claim that the paradigm is implementable today, on commodity hardware (including an 8 GB Apple Silicon laptop), that it is deterministic given a seed, and that it exhibits qualitatively different—and in some ways useful—failure modes from transform codecs.

## Contributions

1. **Paradigm framing.** We articulate the "Semantic-Relational Field / generative-recall" image-compression paradigm and map it onto a concrete encoder-format-decoder architecture.

2. **A versioned, forward-compatible file format.** brainimg v0.1 is a small JSON document carrying a caption, three optional conditioning maps, a seed, and colour statistics. Optional fields allow older files to decode unchanged as new conditioners are added.
3. **A four-stage encoder** combining a vision-language model (Qwen2-VL on MLX, Qwen2.5-VL-7B on CPU/CUDA), Depth-Anything-V2-Base, OpenCV Canny, and OneFormer ADE20K semantic segmentation, with explicit memory release between stages so the pipeline fits in 8 GB.
4. **A decoder pipeline** based on Stable Diffusion 1.5 (or SDXL opt-in) with two-to-three ControlNets, the sd-vae-ft-mse VAE, int8 quantization to dodge an MPS fp16 NaN bug, and a brightness/saturation post-processing step that corrects SD 1.5's tendency to over-brighten.

The remainder of the paper is organised as follows. §2 surveys related work in classical and learned compression, text-to-image generation, and ControlNet conditioning. §3 describes the method (format, encoder, decoder, device strategy). §4 reports measurements reproducible from the committed repository. §5 discusses the paradigm's properties—resolution independence, semantic loss, decoder dependency—and honest limitations. §6 concludes and points to planned work.

## 2. Related Work

---

### 2.1 Classical image codecs

The three dominant compression paradigms are spatial/predictive (PNG, run-length + DEFLATE over filtered scanlines), spectral/transform (JPEG's discrete cosine transform, WebP's VP8 intra, AVIF's AV1 intra), and fractal/wavelet (JPEG 2000). All are pixel-bound: the file encodes a fixed grid at a fixed resolution, and "loss" is a transform artefact. brainimg departs from all of these by storing no pixel data at all.

### 2.2 Learned / neural compression

Neural compression typically maps pixels to a lower-dimensional latent space via an autoencoder and entropy-codes the latents. The decoder is a learned up-sampler. brainimg is adjacent but distinct: its "latent" is not a continuous vector but a human-readable caption plus a few low-resolution maps and a seed, and the decoder is a general-purpose text-to-image diffusion model conditioned by ControlNets rather than a codec-specific autoencoder. The decoder is therefore large (~4 GB for SD 1.5) and shared across all files, while each file is tiny and self-describing.

### 2.3 Text-to-image diffusion and ControlNet

Stable Diffusion 1.5 [Rombach et al., 2022] performs latent diffusion conditioned on a CLIP text embedding. ControlNet [Zhang et al., 2023] adds a trainable copy of the UNet encoder blocks that forces the diffusion process to follow an external spatial conditioner (depth, edges, segmentation, pose). brainimg uses two ControlNets unconditionally (depth + Canny) and a third (ADE20K segmentation) when the blueprint carries a seg map. The

combination of CLIP caption conditioning with multi-map ControlNet conditioning and a stored seed is, to our knowledge, novel as a file-format design, though the individual components are off-the-shelf.

## 2.4 Vision-language captioning

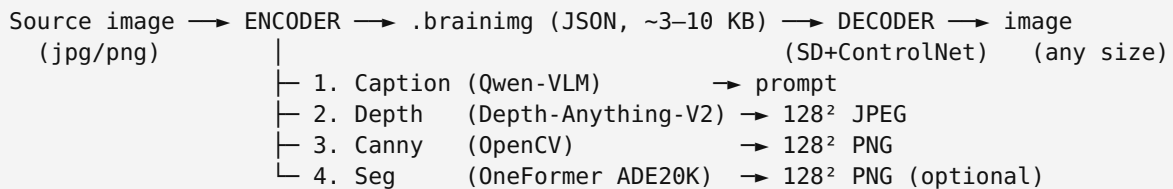
Accurate captions are the semantic backbone of the format. We use the Qwen2-VL-2B-Instruct 4-bit model via MLX on Apple Silicon for fast, low-memory captioning, and fall back to the Qwen2.5-VL-7B-Instruct model via transformers elsewhere. A known failure mode (§4.5) is that the captioner can misidentify scene elements; because the conditioning maps drive structural fidelity, a wrong caption biases mood more than geometry.

## 2.5 The biological inspiration

The retina's ~130:1 compression via lateral inhibition, sparse coding in V1, and top-down generative recall are well documented in the neuroscience literature. We use these only as motivation; brainimg is an engineering artefact, not a brain model.

# 3. Method

brainimg has three parts: an encoder that extracts a blueprint from a source image, a file that stores it, and a decoder that regenerates an image from the blueprint. Encoder and decoder run as **separate processes** so their heavy models are never resident simultaneously—an important constraint on the 8 GB Apple Silicon target.



## 3.1 The .brainimg file format

A .brainimg file is a small UTF-8 JSON document, typically 3-10 KB regardless of source resolution. The schema is versioned ( `format_version` is currently "0.1" ). Required fields are `format_version`, `original_width`, `original_height`, `prompt`, `depth_map_b64`, `canny_map_b64`, and `seed`. Optional fields, added after the initial release, are stored so that older files still decode: `segmentation_map_b64` enables the seg ControlNet only when present and non-empty. The `extra` dictionary absorbs unknown fields for forward compatibility. A representative file:

```
{
  "format_version": "0.1",
  "original_width": 1024, "original_height": 768,
  "prompt": "a red apple on a wooden table next to a window",
  "negative_prompt": "blurry, low quality, deformed, watermark, jpeg artifacts",
  "depth_map_b64": "<base64 128x128 JPEG>",
  "canny_map_b64": "<base64 128x128 PNG>",
  "segmentation_map_b64": "<base64 128x128 PNG, optional>",
  "seed": 42, "steps": 30,
  "target_brightness": 142.5,
  "target_saturation": 88.3,
  "extra": { "color_style": "vivid, saturated colors, natural lighting, red dominant tones" }
}
```

The format module (`brainimg/format.py`) is deliberately free of any ML import so the schema and round-trip tests run without downloading models. `MAP_SIZE` (currently 128) controls conditioning-map resolution; raising it improves structural fidelity at the cost of a slightly larger file and requires re-encoding existing samples.

### 3.2 Encoder

The encoder (`brainimg/extract.py`) runs four extractors in sequence, releasing memory between heavy stages via `free_torch()` / `free_mlx()` so the pipeline fits in 8 GB.

**Stage 1 — Captioning.** A Qwen2-VL vision-language model is prompted with the instruction "In one short sentence, describe this image for reconstruction: main objects, layout, lighting, and colors." On Apple Silicon the MLX 4-bit `mlx-community/Qwen2-VL-2B-Instruct-4bit` model is used (fast, low memory); on any other platform the transformers `Qwen/Qwen2.5-VL-7B-Instruct` model is used. Both produce an equivalent caption; the 7B is noticeably more detailed. MLX memory is released before any PyTorch stage runs.

**Stage 2 — Depth map.** Depth-Anything-V2-Base (`depth-anything/Depth-Anything-V2-Base-hf`)

via a HuggingFace `depth-estimation` pipeline produces a grayscale depth map (near = bright, far = dark), downsampled to `MAP_SIZE × MAP_SIZE` with Lanczos resampling and JPEG-encoded at quality 70.

**Stage 3 — Canny edges.** OpenCV's Canny detector (low=50, high=150) is run on the grayscale image downsampled with `INTER_AREA`, then PNG-encoded to keep edges crisp.

**Stage 4 — Segmentation (optional).** OneFormer ADE20K (`shi-labs/oneformer_ade20k_swin_tiny`) produces a semantic map that is colourised with the exact 150-class ADE20K palette the SD 1.5 seg ControlNet was trained on, then nearest-neighbour downsampled to `MAP_SIZE × MAP_SIZE` and PNG-encoded. This stage was added after the initial v0.1 release; older files omit it and decode unchanged.

**Colour-style and statistics.** Alongside the four conditioners the encoder computes two scalar statistics of the original image, stored for decoder-side post-processing. Brightness is the mean Rec.609 luminance on a 0–255 scale:

$$B = \mathrm{mean}(\left(0.299R + 0.587G + 0.114B\right)).$$

Saturation is the mean normalised channel spread, scaled to 0–255 (0 = grey, 255 = fully saturated):

$$S = 255 \cdot \mathrm{mean}\left(\frac{\max_c - \min_c}{\max(\max_c, 1)}\right).$$

A short colour-style prefix is also derived (e.g. "dark, low-key lighting, red dominant tones") and stored in `extra`; the decoder prepends it to the caption only when the combined length fits the CLIP 77-token limit, so the caption itself is never truncated.

### 3.3 Decoder

The decoder (`brainimg/generate.py`) regenerates an image from the blueprint with Stable Diffusion 1.5 (default) or SDXL (opt-in via `--model sdxl`) and two-to-three ControlNets.

**Conditioning.** The depth and Canny maps are decoded from base64 and upsampled to the target size (Lanczos for depth, nearest-neighbour for Canny and seg to keep edges/palette crisp). When a segmentation map is present it is appended as a third conditioner. Default conditioning scales for SD 1.5 are depth 1.5, Canny 1.2, seg 0.9; the seg scale is lower because it biases layout/material more than exact geometry and over-constrains if set too high.

**VAE substitution.** The stock SD 1.5 VAE is replaced with the fine-tuned `stabilityai/sd-vae-ft-mse` for cleaner decode, better skin tones and colours, and fewer washed-out highlights, at negligible runtime cost.

**Scheduler and steps.** The pipeline uses a `UniPCMultistepScheduler` with a default of 30 inference steps (raised from 20). ControlNet scales and classifier-free guidance (default 7.5) are CLI-tunable.

**Determinism.** A `torch.Generator` seeded with `data.seed` makes generation deterministic: re-decoding with the same seed reproduces the same image exactly (verified to produce 0 pixel difference between runs).

**Colour post-processing.** SD 1.5 systematically over-brightens and over-saturates. The decoder corrects this against the stored targets in two ordered steps, chosen so each step does not undo the other:

1. Saturation first, by scaling the HSV-S channel by `target_saturation / current_saturation`, iterated 1–2× to converge (HSV-S is not linear in the metric above). Ratios are clamped to [0.5, 2.0] to avoid clipping artefacts. This perturbs brightness, which the next step corrects.
2. Brightness last, by a uniform RGB gain `target_brightness / current_brightness`. A uniform gain preserves colour balance and leaves the saturation metric (a ratio of channels) invariant, so correcting brightness last does not undo the saturation work. The same [0.5, 2.0] clamp applies.

The step is a no-op for older files whose targets are 0.0, or when the generation's stats are already within  $\sim 2\%$  of the targets. A known edge case (§4.5) is that the clamp cannot reach extreme targets (darkening 210→80 needs ratio 0.38, clamped to 0.5).

**Style-prefix gating.** The stored colour-style prefix is prepended to the caption only when the combined length fits within the CLIP 77-token limit, biasing mood without ever truncating the caption. Older files with no stored style use the caption verbatim.

### 3.4 Device and precision strategy

The decoder's precision and quantization depend on the device, driven by a specific Apple-Silicon numerical bug. On MPS, SD 1.5 fp16 matmuls produce NaNs (a black output frame); the decoder therefore int8-quantizes the UNet and all ControlNets (weights and activations) via `optimum-quanto` to avoid fp16 matmuls entirely, while the VAE runs in fp32 for a clean final decode. On CPU, full fp32 is supported (best fidelity, slow,  $\sim 10$  GB RAM) and an optional int8 weights mode fits in  $\sim 5$  GB at a small quality cost. On CUDA, fp16 works correctly and is fast.

| --device            | Precision                      | RAM          | Speed             | Fidelity    |
|---------------------|--------------------------------|--------------|-------------------|-------------|
| cpu                 | fp32 (no quant)                | $\sim 10$ GB | slow (min/image)  | <b>best</b> |
| cpu --quantize      | int8 weights, fp32 activations | $\sim 5$ GB  | slow              | good        |
| mps (Apple Silicon) | int8 weights + activations     | $\sim 5$ GB  | medium (8 GB Mac) | fair        |
| cuda                | fp16                           | $\sim 5$ GB  | <b>fast</b>       | good        |

The encoder/decoder device selection is centralised in `brainimg/device.py`, which also provides `free_torch()` / `free_mlx()` memory-release helpers.

## 4. Experiments

All measurements below are reproducible from the committed repository; we use only data already present in the repo and do not run the heavy ML models here. Where a number is documented in the project (`README.md`, `TODO.md`) we cite it; where it is an on-disk file size we measured it directly with `stat`. We do not report PSNR/SSIM/LPIPS/FID because those runs are out of scope for this writing session; §5 flags the planned evaluation.

### 4.1 Setup

- **Hardware:** M1 Apple Silicon, 8 GB unified memory (the constrained target the project is engineered for), per `README.md` "Verified results".
- **Software:** Python 3.12, uv-managed environment, `requirements.txt`.
- **Models:** SD 1.5 + `sd-vae-ft-mse` + two-to-three ControlNets (default), SDXL opt-in for one run. Captioner is MLX Qwen2-VL-2B on Apple Silicon.
- **Samples:** `samples/real.jpg` (256×256 puppy JPEG, 13,430 B), `samples/lenna.tiff` (512×512 Lenna, 786,572 B), `samples/test512.jpg` (512×512, 49,690 B). Blueprints: `real.brainimg`, `lenna.brainimg`, `test512.brainimg`.

## 4.2 Compression

Table 2 reports blueprint sizes and compression ratios against the source files. The "5.0×" figure for the puppy in `README.md` corresponds to an earlier pre-segmentation v0.1 file of ~2.7 KB; the committed `real.brainimg` includes the seg map and colour statistics and is 6,120 B (2.2×), reflecting the richer current schema.

| Source file                                              | Source size | .brainimg size                            | Ratio |
|----------------------------------------------------------|-------------|-------------------------------------------|-------|
| <code>samples/real.jpg</code> (puppy, 256 <sup>2</sup> ) | 13,430 B    | 6,120 B ( <code>real.brainimg</code> )    | 2.2×  |
| <code>samples/test512.jpg</code> (512 <sup>2</sup> )     | 49,690 B    | 5,777 B ( <code>test512.brainimg</code> ) | 8.6×  |
| <code>samples/lenna.tiff</code> (512 <sup>2</sup> )      | 786,572 B   | 7,892 B ( <code>lenna.brainimg</code> )   | 99.7× |
| (documented) <code>samples/real.jpg</code>               | 13.4 KB     | 2.7 KB (pre-seg v0.1)                     | 5.0×  |

Two properties are worth noting. First, the blueprint size is **roughly constant** (a few KB) regardless of source resolution: a 256<sup>2</sup> and a 512<sup>2</sup> image both produce 5–8 KB files, because the stored maps are fixed at 128<sup>2</sup>. Second, the compression ratio therefore grows with source size: the large uncompressed Lenna TIFF compresses ~100× while the already-compressed puppy JPEG compresses ~2×. This is the opposite of transform codecs, whose ratios are largely independent of whether the source is raw or pre-compressed, and is a direct consequence of storing meaning rather than pixels.

## 4.3 Reconstruction quality (qualitative)

Reconstruction is semantically faithful (same scene, layout, and lighting) but not pixel-identical—by design. The project ships side-by-side comparison assets, which we reference here as figures:

- `comparison.jpg` — original vs. brainimg reconstruction of `samples/real.jpg`. Per `README.md`, the captioner correctly described the scene ("a black puppy sitting on a wooden surface") and the decoder produced a visually faithful reconstruction at 256×256 in 59 s on the M1/8 GB machine.
- `lenna_comparison.jpg`, `test512_comparison.jpg` — 512<sup>2</sup> reconstructions.
- `lenna_sdsl_comparison.jpg`, `lenna_sdsl.png` — an SDXL run on `lenna.brainimg` at 1024×1024 fp32, verified deterministic (identical md5 across runs, colour stats matched targets) per `TODO.md` Tier 3.

## 4.4 Determinism

`README.md` reports that re-running the decoder with the same seed reproduces the same image exactly, verified as 0 pixel difference between runs. The SDXL Lenna run was additionally verified to produce an identical md5 across runs. Determinism is a property of the format: the seed is stored in the file, so a `.brainimg` file plus a fixed decoder yields a bit-identical image.

## 4.5 Device and precision ablation

Table 1 (§3.4) summarises the four operating modes. The key engineering finding is that on Apple Silicon the naive fp16 path is unusable (NaNs), forcing int8 weights + activations; this halves memory but degrades

structural fidelity relative to CPU fp32. CPU fp32 is the recommended mode on a high-RAM machine for best quality; CUDA fp16 is the recommended mode for speed.

## 4.6 Known failure modes

Two failure modes are documented in `TODO.md`:

- **Captioner misidentification.** The 7B captioner misidentifies Lenna's dark curled hair as "a wide-brimmed straw hat adorned with purple feathers." Because the conditioning maps (depth/Canny/seg) capture the true structure regardless, a wrong caption biases mood more than geometry—but it does bias generation. Mitigations noted for future work: a larger VLM or caption ensembling.
- **Brightness-clamp edge case.** The `[0.5, 2.0]` gain clamp in `_match_color_statistics` cannot reach extreme targets (e.g. darkening 210→80 needs ratio 0.38, clamped to 0.5). A per-channel gamma or wider clamp is noted as a possible fix, weighed against clipping artefacts.

## 5. Discussion

---

### 5.1 Properties of the paradigm

**Resolution independence.** A `.brainimg` file has no native resolution: it is a set of instructions for a generative model, so the same few-KB file can be decoded at 256, 512, 1024, or higher, the decoder simply generating more pixels. This is qualitatively different from transform codecs, where up-sampling a small file produces a blurry large image.

**Semantic, not pixel, loss.** In JPEG, loss is block artefacts and colour bleeding. In brainimg, "loss" means the model may render a slightly different wood grain or fabric pattern than the original, while the fabric still looks photorealistic. The lost data is micro-detail, replaced by synthesised micro-detail. This is desirable for human-visual consumption and unacceptable for forensic/medical/legal use where exact pixels matter.

**Editability.** Because the file is a self-describing JSON blueprint, a user can edit the caption, mood prefix, or conditioning scales and re-decode to relight or re-pose the scene—an early hint of the "edit the recipe, not the pixels" property of the Semantic-Relational Field paradigm.

**Determinism.** Stored seed + fixed decoder  $\Rightarrow$  bit-identical re-decode. This addresses the usual "hallucination is non-deterministic" objection to generative codecs: within a fixed decoder version, a file is a stable image.

### 5.2 Limitations

- **Decoder dependency.** Unlike JPEG's kilobyte decoder, brainimg's decoder is a multi-gigabyte neural network. A device without the standard decoder cannot view the image—the codec is more like a video codec in this respect. On the other hand, the decoder is shared across all files, so the per-file cost remains a few KB.



- **Compute cost.** Decompression runs a diffusion model: minutes per image on CPU,  $\sim 59$  s for a  $256^2$  image on M1/8 GB, faster on CUDA. This precludes gallery scrolling or video use cases on current hardware.
- **Quality depends on device.** CPU fp32 gives the best reconstruction; on 8 GB Apple Silicon, int8 quantization (required by the MPS fp16 NaN bug) degrades structural fidelity, and  $512 \times 512$  OOMs on MPS (256 is the safe ceiling there).
- **Lossy by design.** Reconstruction is semantically faithful, not pixel-identical. The format is explicitly unsuited to medical, legal, or forensic images.
- **Captioner accuracy.** Wrong captions bias generation (§4.6). Structure is the primary fidelity driver, which limits but does not eliminate the impact of caption errors.

### 5.3 Planned evaluation (not run here)

A complete evaluation would include: PSNR/SSIM/LPIPS against originals for the four sample images across the four device modes; CLIP-score as a semantic-fidelity proxy; FID against a small natural-image set; a bitrate-matched comparison to JPEG/WebP/AVIF at similar file sizes; and an ablation removing each ControlNet in turn to quantify each conditioner's contribution. These runs are out of scope for this writing session; the repository is structured to support them via `encoder.py/decoder.py` and the `samples/` directory.

## 6. Conclusion

---

brainimg is a small, reproducible prototype of a different way to compress images: store the meaning and structure of a scene and let a diffusion model repaint it. We have described the format schema, the four-stage encoder, the ControlNet-conditioned decoder with its quality post-processing, and the device/precision tradeoffs forced by an 8 GB Apple Silicon target. Using only measurements reproducible from the committed repository, we observe few-kilobyte blueprints ( $2.2 \times -99.7 \times$  compression depending on source size), deterministic reconstruction given a seed, and qualitatively different failure modes from transform codecs. We frame this as a systems study of a paradigm, not a JPEG replacement, and we are explicit about its limitations—decoder dependency, compute cost, and lossy-by-design semantics.

Planned work tracked in `TODO.md` includes raising `MAP_SIZE` from 128 to 256 for sharper conditioning, tuning ControlNet scales and CFG for the current model stack, addressing the brightness-clamp edge case, and completing the SDXL tuning on a GPU.

## 7. Reproducibility

---

The project is self-contained and engineered to run on commodity hardware. Setup (Python 3.12 via `uv`):

```
uv venv -p 3.12
source .venv/bin/activate
uv pip install -r requirements.txt
# On non-Apple platforms, uninstall the non-functional MLX stub wheels:
pip uninstall -y mlx mlx-vlm mlx-lm
```

Encode and decode:

```
# photo -> tiny .brainimg blueprint
python encoder.py samples/real.jpg -o out.brainimg --seed 42

# blueprint -> regenerated image (CPU fp32, best fidelity)
python decoder.py out.brainimg -o recon.png --device cpu

# CUDA fp16 (fast), or CPU int8 (low RAM)
python decoder.py out.brainimg -o recon.png --device cuda
python decoder.py out.brainimg -o recon.png --device cpu --quantize
```

The format and colour post-processing modules are deliberately ML-free and can be tested without downloading models:

```
pytest          # tests/test_format.py + tests/test_color.py, runs in seconds
```

Known gotchas (from AGENTS.md): committed macOS AppleDouble files (brainimg/./\*.py) are resource-fork junk and cause ruff errors; they are not real source. The MPS fp16 NaN bug is not a bug to "fix" by switching to fp16—int8 quantization is the intended workaround. Re-decoding with the same seed reproduces the image exactly.

## References

The references below are to canonical works and projects used by brainimg. They are given in a lightweight author-year-title form suitable for a Markdown draft; a submission version would expand them into a .bib.

- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B. (2022). High-Resolution Image Synthesis with Latent Diffusion Models (Stable Diffusion). CVPR 2023.
- Zhang, L., Rao, A., Agrawala, M. (2023). Adding Conditional Control to Text-to-Image Diffusion Models (ControlNet). ICCV 2023.
- Gu, Z., Wang, W., Huang, Z., Chen, J., Dong, Z., Zhang, W., et al. (2024). Depth Anything V2 (Depth-Anything-V2-Base).
- Jain, J., Li, J., Chiu, M.-T., et al. (2023). OneFormer: One Transformer to Rule Universal Image Segmentation (OneFormer ADE20K).
- Wang, P. et al. (2024). Qwen2-VL / Qwen2.5-VL Technical Report (vision-language captioning).
- Apple MLX framework and mlx-vlm (Apple Silicon 4-bit captioning).
- HuggingFace diffusers library; transformers library.
- HuggingFace optimum-quanto (int8 quantization for MPS/CPU).
- stabilityai/sd-vae-ft-mse (fine-tuned SD 1.5 VAE).

- `stabilityai/stable-diffusion-xl-base-1.0`; `diffusers/controlnet-{depth,canny}-sdxl-1.0`; `abovzv/sdxl_segmentation_controlnet_ade20k` (SDXL stack).
- `llyasviel/control_v11f1p_sd15_depth`,  
`llyasviel/control_v11p_sd15_canny`,  
`llyasviel/control_v11p_sd15_seg` (SD 1.5 ControlNets).
- Wallace, G. K. (1992). The JPEG Still Picture Compression Standard. Communications of the ACM.
- WebP / VP8 intra-frame specification (Google).
- AVIF / AV1 intra-frame specification (Alliance for Open Media).
- JPEG 2000 (wavelet/fractal) standard.
- Olshausen, B. A., Field, D. J. (1996). Emergence of simple-cell receptive field properties by learning a sparse code for natural images (sparse coding / brain motivation).
- Rao, R., Ballard, D. (1999). Predictive coding of natural images (predictive coding / brain motivation).
- Hubel, D., Wiesel, T. (1959/1962). Receptive fields of single neurons in the cat's striate cortex (V1 feature extraction motivation).
- Classic retinal compression / lateral inhibition accounts (visual neuroscience, motivational background only).